SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

Department of Computer Science Engineering

Course Code: 23CS002PC304
Course Title: AI Assisted Coding
Year/Sem: III / II
Assignment Number: 13.4 / 24

Name: O.Narasimnha Raju
HT No: 2303A53034
Batch: 46

Lab Objectives

- Refactor complex legacy code using AI assistance
- Improve modularity and structure
- Optimize performance
- Enhance readability and maintainability

Task 1: Removing Duplicate Logic

Legacy Code

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")

marks = 72
```

```
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

Refactored Code

```
def calculate_grade(marks):
    if marks >= 90:
        return "Grade A"
    elif marks >= 75:
        return "Grade B"
    else:
        return "Grade C"

print(calculate_grade(85))
print(calculate_grade(72))
```

Task 2: Modular Programming

```
def get_input():
    return int(input("Enter number: "))

def calculate_square(n):
    return n * n

def display_result(result):
    print("Square:", result)

num = get_input()
result = calculate_square(num)
display_result(result)
```

Task 3: Optimizing List Processing

Legacy Code

```
nums = [1, 2, 3, 4, 5]
squares = []
```

```
for n in nums:
    squares.append(n * n)

print(squares)
```

Optimized Code

```
nums = [1, 2, 3, 4, 5]
squares = [n * n for n in nums]

print(squares)
```

Task 4: Improving Readability

```
def calculate_area(length, width):
    return length * width

print(calculate_area(10, 5))
```

Conclusion

Advanced refactoring improves code structure, performance, and readability.

AI tools significantly reduce development effort by suggesting optimized solutions.